ServiceNow Role-Based Access Control (RBAC) & Workflow Automation Simulator

Version 1.0.0 **Platform** Vite / JavaScript (ES Modules) **Classification** Internal Technical Reference

1. Executive Summary

The ServiceNow RBAC & Workflow Simulator is a browser-based, client-side application built to reproduce core ServiceNow platform behaviors — Access Control List (ACL) enforcement, role-based record visibility, and Flow Designer-style workflow automation — in a self-contained sandbox. The application allows a reviewer to impersonate different user personas, interact with Project and Project Task records, and observe how security rules and automated flows respond in real time.

The system is implemented as a single-page application using vanilla JavaScript with a lightweight Vite build pipeline, persists state via browser LocalStorage, and includes an embedded automated test suite that validates the RBAC and workflow logic against six defined test cases.

2. Project Overview

2.1 Purpose

The simulator demonstrates, in an interactive and auditable way, how ServiceNow-style security constructs (Users, Groups, Roles, and ACLs) govern access to custom application tables, and how Flow Designer-style automation reacts to record changes such as task creation and SLA breaches.

2.2 Technology Stack

Component	Technology / Approach
Build Tooling	Vite 5.x (dev server, bundling, and preview)
Application Logic	Vanilla JavaScript (ES Modules) — app.js
Markup & Styling	index.html / style.css (custom design system)

Component	Technology / Approach
Data Persistence	Browser LocalStorage (projects, tasks, notifications, flow metrics)
Testing	Custom assertion-based test runner — test_suite.js
Iconography	Font Awesome solid icon set

2.3 Application Modules

- **Projects** — Project record list and detail management (table: *u_project*).
- **Project Tasks** — Task list, assignment, state, priority, and due-date management (table: *u_project_task*).
- **System Administration** — Users, Groups, Roles, and ACL configuration views (admin-only).
- **Flow Designer View** — Visual representation of the active auto-assignment/escalation flow and manual SLA trigger.
- **Update Set XML Preview** — Developer-tools view rendering the simulated ServiceNow XML serialization.
- **Test Runner Panel** — In-app execution and reporting of the automated RBAC and workflow test suite.

3. System Architecture

The application is architected around a single in-memory system object, `ServiceNowSystem`, instantiated once at page load. This object owns all reference data (users, groups, roles), transactional data (projects, tasks), and system metrics (flow run counters, escalation counters, notification queue), and exposes the access-control and workflow-automation methods consumed by the UI layer.

3.1 Core Data Model

Entity	Description
users	System user records with id, username, name, email, roles[], and groups[].
groups	Named groups mapping to one or more roles and member user IDs.
roles	Role definitions: *admin*, *u_project_manager*, *u_team_member*.
projects (u_project)	Project records with number, name, manager, status, description.
tasks (u_project_task)	Task records with assignment, state, priority, due date, and escalation flag.

Entity	Description
notifications	Simulated sys_email queue populated by workflow actions.

3.2 State Persistence

Projects, tasks, notifications, and workflow counters (flow run count, escalation count) are serialized to browser LocalStorage on every mutating action via a central `saveDatabase()` routine, allowing session state to survive page reloads within the same browser.

4. Role-Based Access Control (RBAC) Design

Access control is evaluated centrally through `hasAccessForUser(username, table, operation, record)`, which mirrors ServiceNow's ACL evaluation model: role-based table access combined with row-level condition checks where applicable.

4.1 Defined Roles

Role	Description	Assigned To
admin	Unrestricted system access; overrides all other ACL checks.	System Administrator
u_project_manager	Full CRUD access to Projects and Project Tasks.	Alice (alice.pm)
u_team_member	Row-restricted read/write access to assigned tasks only.	Bob (bob.member)
(none / guest)	No access to any application table.	Guest User

4.2 Access Control Matrix

Table	Operation	Project Manager	Team Member	Guest / No Role
u_project	Create / Read / Write / Delete	Allowed	Denied	Denied
u_project_task	Create / Delete	Allowed	Denied	Denied
u_project_task	Read / Write (own record)	Allowed	Allowed	Denied
u_project_task	Read / Write (other's record)	Allowed	Denied	Denied
sys_user / sys_user_group / sys_user_role / sys_security_acl	All	Denied (admin only)	Denied	Denied
sys_hub_flow	Read	Allowed	Denied	Denied

Row-level enforcement on Project Tasks is the most security-critical rule in the system: a Team Member is granted read/write access only when the record's `assigned_to` field matches their own user id, replicating ServiceNow's record-scoped ACL condition scripts.

5. Workflow Automation (Flow Designer Simulation)

The simulated flow, `u_project_task_assignment_escalation_flow`, executes on Project Task creation and performs three sequential actions, in addition to a separately triggerable SLA breach evaluation job.

5.1 Trigger: Task Creation (INSERT_ACTION)

- **Action 1 — Auto-Assignment:** if the new task's `assigned_to` field is empty, the flow automatically assigns it to the default team member (Bob).
- **Action 2 — State Transition:** if the task state is *Open*, it is automatically transitioned to *Work in Progress*.
- **Action 3 — Notification:** an assignment email is generated and queued to the assignee, and a bell alert is raised in the UI.

5.2 SLA Breach Evaluation Job

The SLA job (manually triggerable from the UI or run automatically as part of the test suite) iterates all open tasks and, for any task whose due date has passed and which is not already escalated, performs the following:

- Sets `u_escalated = true` on the task record.
- Raises task priority to *P1 — Critical*.
- Increments the system-wide escalation counter.
- Dispatches an escalation notification email to the Project Manager (Alice).

6. Automated Test Suite & Validation Results

An embedded test module (`test_suite.js`) validates the RBAC and workflow logic through six assertion-based test cases, executed directly against the live application state via the in-app *Run Security Tests* control.

#	Test Case	Validation Criteria	Result
1	User & Role Association	Confirms Alice holds u_project_manager and Bob holds u_team_member.	PASS
2	Project Table ACL Policies	Confirms full CRUD access for Project Managers and complete denial for Team Members.	PASS
3	Task Table ACLs (Project Manager)	Confirms full CRUD access to Project Tasks for the Project Manager role.	PASS
4	Task Table ACLs (Team Member)	Confirms Team Members may only read/write their own assigned tasks, and cannot create or delete.	PASS
5	Workflow Auto-Assignment & State Change	Confirms task creation triggers auto-assignment, state transition, flow logging, and notification dispatch.	PASS
6	Workflow Overdue SLA Escalation	Confirms overdue tasks are escalated to Critical priority, flagged, and the Project Manager is notified.	PASS

Total Test Cases	6	Passed	6	Failed	0

Result note: all six defined assertions passed under standard evaluation conditions at the time of review. Test outcomes are generated live within the application session and are not persisted between runs.

7. Security Observations & Considerations

As a client-side educational simulator, several design characteristics are worth noting for anyone extending this project toward a production context:

- **Client-side enforcement only:** all ACL checks execute in the browser; there is no server-side authority validating requests, which is appropriate for simulation but would require a backend enforcement layer in production.

- **LocalStorage persistence:** application state is stored unencrypted in the browser and is scoped to the local device/browser profile only.
- **Impersonation model:** the UI allows switching the active user context directly, which is intentional for demonstrating RBAC behavior but would need to be replaced with authenticated session handling in a live deployment.
- **Admin override:** the admin role bypasses all table and row-level checks, consistent with ServiceNow's native administrator behavior.

8. Conclusion

The ServiceNow RBAC & Workflow Simulator successfully demonstrates the core mechanics of role-based access control and Flow Designer-style automation in a self-contained, testable environment. All six automated validation checks pass against the current build, confirming that role-based table access, row-level task restrictions, auto-assignment on creation, and SLA-driven escalation behave as specified. The project is well suited as a training, demonstration, or reference implementation for ServiceNow governance concepts.

— End of Report —